

第四章 贪心算法

Greedy

!!贪心不一定正确(0-1背包), 需要证明

4.1 活动安排问题

4.2 贪心算法的基本要素 (分数背包)

4.3 最优装载

4.4 哈夫曼编码

4.6 最小生成树

应用：最优装载

最优装载问题

- 有一批集装箱要装上一艘载重量为 c 的轮船。其中集装箱 i 的重量为 w_i 。最优装载问题要求确定在装载体积不受限制的情况下，将尽可能多的集装箱装上轮船。

最优装载问题

- 问题可形式化描述为
- 其中变量 $x_i=0$ 表示不装入集装箱 i , $x_i=1$ 表示装入集装箱 i 。

$$\max \sum_{i=1}^n x_i$$

$$\sum_{i=1}^n w_i x_i \leq c$$

$$x_i \in \{0,1\}, 1 \leq i \leq n$$

问题

- 贪心选择依据是什么？

分析

- 最优装载问题可用贪心算法求解。
采用重量最轻者先装的贪心选择策略，可产生最优装载问题的最优解。

最优装载问题

- 从剩下的货箱中，选择重量最小的货箱。这种选择次序可以保证所选的货箱总重量最小，从而可以装载更多的货箱。
- 根据这种贪婪策略，首先选择最轻的货箱，然后选次轻的货箱，如此下去直到所有货箱均装上船或船上不能再容纳其他任何一个货箱。

最优装载问题

$[w_1, \dots, w_8] = [100, 200, 50, 90, 150, 50, 20, 80]$,
 $c = 400$ 。

利用贪婪算法时，所考察货箱的顺序为7, 3, 6, 8, 4, 1, 5, 2。

货箱7, 3, 6, 8, 4, 1的总重量为390个单位且已被装载，剩下的装载能力为10个单位，小于剩下的任何一个货箱。

在这种贪婪解决算法中得到 $[x_1, \dots, x_8] = [1, 0, 1, 1, 0, 1, 1, 1]$ 且 $\sum x_i = 6$ 。

最优装载问题

```
void ContainerLoading(int x[], float w[], float c, int n)
{
    Sort(w, t, n); //此时,  $w[i] \leq w[i+1]$ 
    for(int i = 1; i <= n; i++) //初始化x
        x[i] = 0;
    for(i = 1; i <= n && w[i] <= c; i++) {
        x[i] = 1;
        c -= w[i];
    } // 剩余容量
}
```

算法分析

- 贪心选择性质
 - 可以证明最优装载问题具有贪心选择性质。 +
- 最优子结构性质
 - 最优装载问题具有最优子结构性质。
- 算法复杂性
 - 算法主要计算量在于将集装箱依其重量从小到大排序，故算法所需的计算时间为 $O(n\log n)$ 。

最优装载

最优装载

- 输入: n 物品重 $W[1:n]$,
背包容量 C
- 输出: 装包使得件数最多.
- 每件物品只能取或不取

$$\max \sum_{i=1}^n x_i$$

$$\sum_{i=1}^n W_i x_i \leq C$$

$$x_i \in \{0,1\}, 1 \leq i \leq n$$

- $n=8$, $[w_1, \dots, w_8] = [100, 200, 50, 90, 150, 50, 20, 80]$,
 $C=400$ 从剩下的货箱中, 选择重量最小的货箱

- 贪心选择性质: 最优解可以包含重量最小的
- OSP: 最优解 $([1:n], C)$ 去掉重量最小 $([2:n], C-W[1])$ 仍是最优解

最优装载贪心法证明

设集装箱已依其重量从小到大排序, $(x_1, x_2, \dots, x_n)$ 是最优装载问题的一个最优解。又设 $k = \min \{i \mid x_i = 1\}$ 。易知, 如果给定的最优装载问题有解, 则 $1 \leq k \leq n$ 。

(1) 当 $k = 1$ 时, $(x_1, x_2, \dots, x_n)$ 是一个满足贪心选择性质的最优解。

(2) 当 $k > 1$ 时, 取 $y_1 = 1; y_k = 0; y_i = x_i, 1 < i \leq n, i \neq k$, 则,

$$\sum_{i=1}^n w_i y_i = w_1 - w_k + \sum_{i=1}^n w_i x_i \leq \sum_{i=1}^n w_i x_i \leq c$$

因此, $(y_1, y_2, \dots, y_n)$ 是所给最优装载问题的一个可行解。

另一方面, 由 $\sum_{i=1}^n y_i = \sum_{i=1}^n x_i$ 知, $(y_1, y_2, \dots, y_n)$ 是一个满足贪心选择性质的最优解。所以, 最优装载问题具有贪心选择性质。

设 $(x_1, x_2, \dots, x_n)$ 是最优装载问题的一个满足贪心选择性质的最优解, 则易知, $x_1 = 1$, $(x_2, \dots, x_n)$ 是轮船载重量为 $c - w_1$ 且待装船集装箱为 $\{2, 3, \dots, n\}$ 时相应最优装载问题的一个最优解。也就是说, 最优装载问题具有最优子结构性质。

最优装载贪心法证明

设集装箱已依其重量从小到大排序, $(x_1, x_2, \dots, x_n)$ 是最优装载问题的一个最优解。又设

$$k = \min_{1 \leq i \leq n} \{i \mid x_i = 1\}$$

易知, 如果给定的最优装载问题有解, 则 $1 \leq k \leq n$ 。

(1) 当 $k = 1$ 时, $(x_1, x_2, \dots, x_n)$ 是一个满足贪心选择性质的最优解。

(2) 当 $k > 1$ 时, 取 $y_1 = 1$; $y_k = 0$; $y_i = x_i$, $1 < i \leq n, i \neq k$, 则

$$\sum_{i=1}^n w_i y_i = w_1 - w_k + \sum_{i=1}^n w_i x_i \leq \sum_{i=1}^n w_i x_i \leq c$$

因此, $(y_1, y_2, \dots, y_n)$ 是所给最优装载问题的一个可行解。

另一方面, 由 $\sum_{i=1}^n y_i = \sum_{i=1}^n x_i$ 知, $(y_1, y_2, \dots, y_n)$ 是一个满足贪心选择性质的最优解。所以, 最优装载问题具有贪心选择性质。

最优装载贪心法证明(续)

设 $(x_1, x_2, \dots, x_n)$ 是最优装载问题的一个满足贪心选择性质的最优解, 则易知, $x_1 = 1$, $(x_2, \dots, x_n)$ 是轮船载重量为 $c - w_1$ 且待装船集装箱为 $\{2, 3, \dots, n\}$ 时相应最优装载问题的一个最优解。也就是说, 最优装载问题具有最优子结构性质。

第四章 贪心算法

Greedy

!!贪心不一定正确(0-1背包), 需要证明

4.1 活动安排问题

4.2 贪心算法的基本要素 (分数背包)

4.3 最优装载

4.4 哈夫曼编码

4.6 最小生成树

应用：哈夫曼编码

哈夫曼编码

哈夫曼编码是广泛地用于数据文件压缩的十分有效的编码方法。其压缩率通常在20%~90%之间。哈夫曼编码算法用字符在文件中出现的频率表来建立一个用0，1串表示各字符的最优表示方式。

给出现频率高的字符较短的编码，出现频率较低的字符以较长的编码，可以大大缩短总码长。

哈夫曼编码

1、前缀码

对每一个字符规定一个0, 1串作为其代码，并要求任一字符的代码都不是其它字符代码的前缀。这种编码称为前缀码。

哈夫曼编码

字符	a	b	C	d	e	F
频率	45	13	12	16	9	5
变长码	0	101	100	111	1101	1100

例如：对于给定的0, 1串001011101可唯一地分解为0, 0, 101, 1101，其译码为a, a, b, e。

不同的哈夫曼编码方式

字符	a	b	C	d	e	F
频率	45	13	12	16	9	5
变长码	0	101	100	111	1101	1100

字符	a	b	C	d	e	F
频率	45	13	12	16	9	5
变长码	101	0	100	111	1101	1100

哈夫曼编码

编码的前缀性质可以使译码方法非常简单。

表示**最优前缀码**的二叉树总是一棵**完全二叉树**，
即树中任一结点都有2个儿子结点。

平均码长定义为： $f(c)$ 为 c 在数据文件中出现的频率， $d_T(c)$ 为 c 在树 T 中的深度。

$$B(T) = \sum_{c \in C} f(c) d_T(c)$$

使平均码长达到最小的前缀码编码方案称为给定编码字符集 C 的**最优前缀码**。

哈夫曼编码

2、构造哈夫曼编码

哈夫曼提出构造最优前缀码的贪心算法，由此产生的编码方案称为**哈夫曼编码**。

哈夫曼算法以自底向上的方式构造表示最优前缀码的二叉树T。

算法以 $|C|$ 个叶结点开始，执行 $|C|-1$ 次的“合并”运算后产生最终所要求的树T。

哈夫曼编码

在书上给出的算法huffmanTree中，编码字符集中每一字符 c 的频率是 $f(c)$ 。以 f 为键值的优先队列 Q 用在贪心选择时有效地确定算法当前要合并的2棵具有最小频率的树。一旦2棵具有最小频率的树合并后，产生一棵新的树，其频率为合并的2棵树的频率之和，并将新树插入优先队列 Q 。经过 $n-1$ 次的合并后，优先队列中只剩下一棵树，即所要求的树 T 。

例子

字符	a	b	C	d	e	F
频率	45	13	12	16	9	5
变长码	0	101	100	111	1101	1100

字符	a	b	c	d	e	F
频率	45	13	12	16	9	5

F: 5

e: 9

d: 16

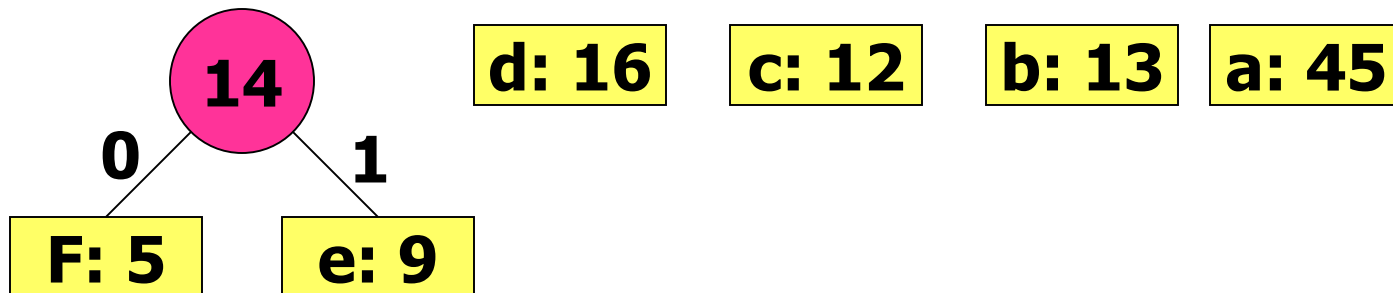
c: 12

b: 13

a: 45

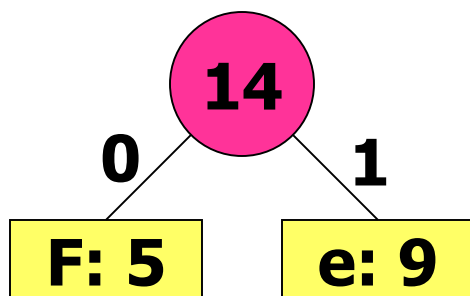
例子(续1)

F: 5 **e: 9** **d: 16** **c: 12** **b: 13** **a: 45**

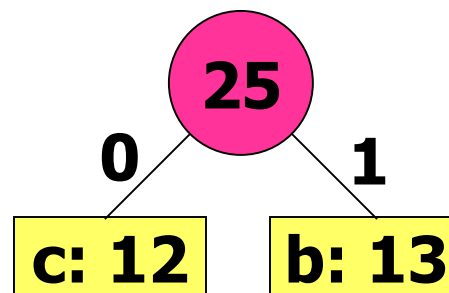


例子(续2)

F: 5 **e: 9** **d: 16** **c: 12** **b: 13** **a: 45**



d: 16



a: 45

例子(续3)

F: 5

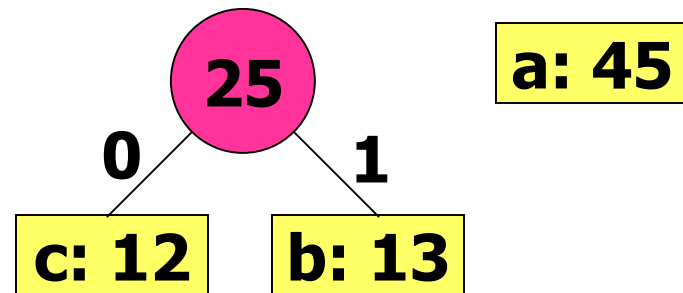
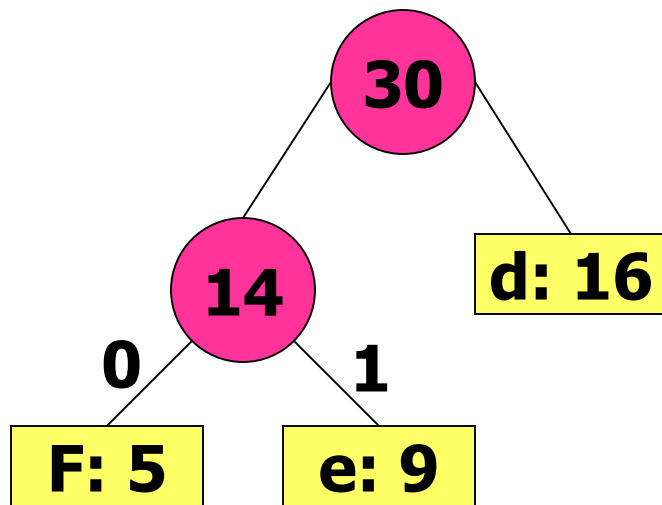
e: 9

d: 16

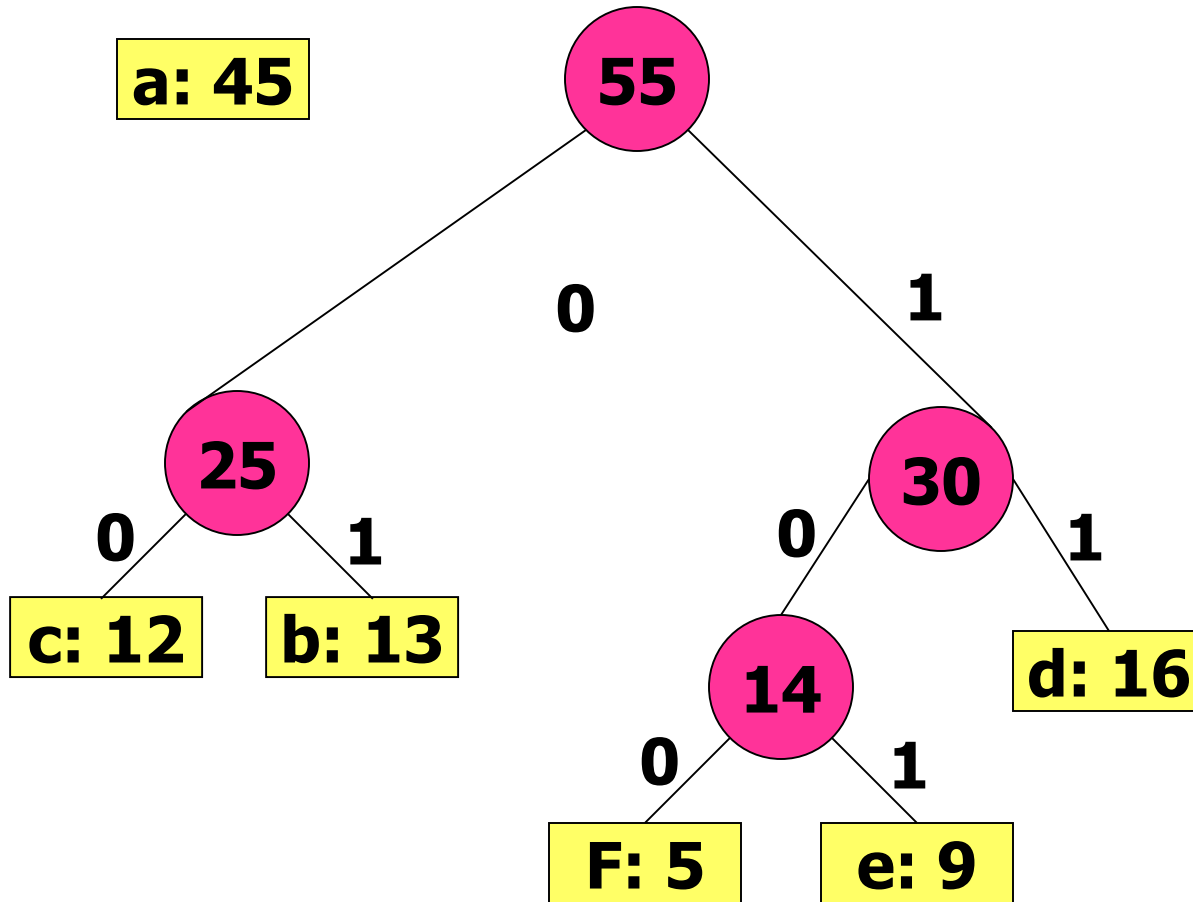
c: 12

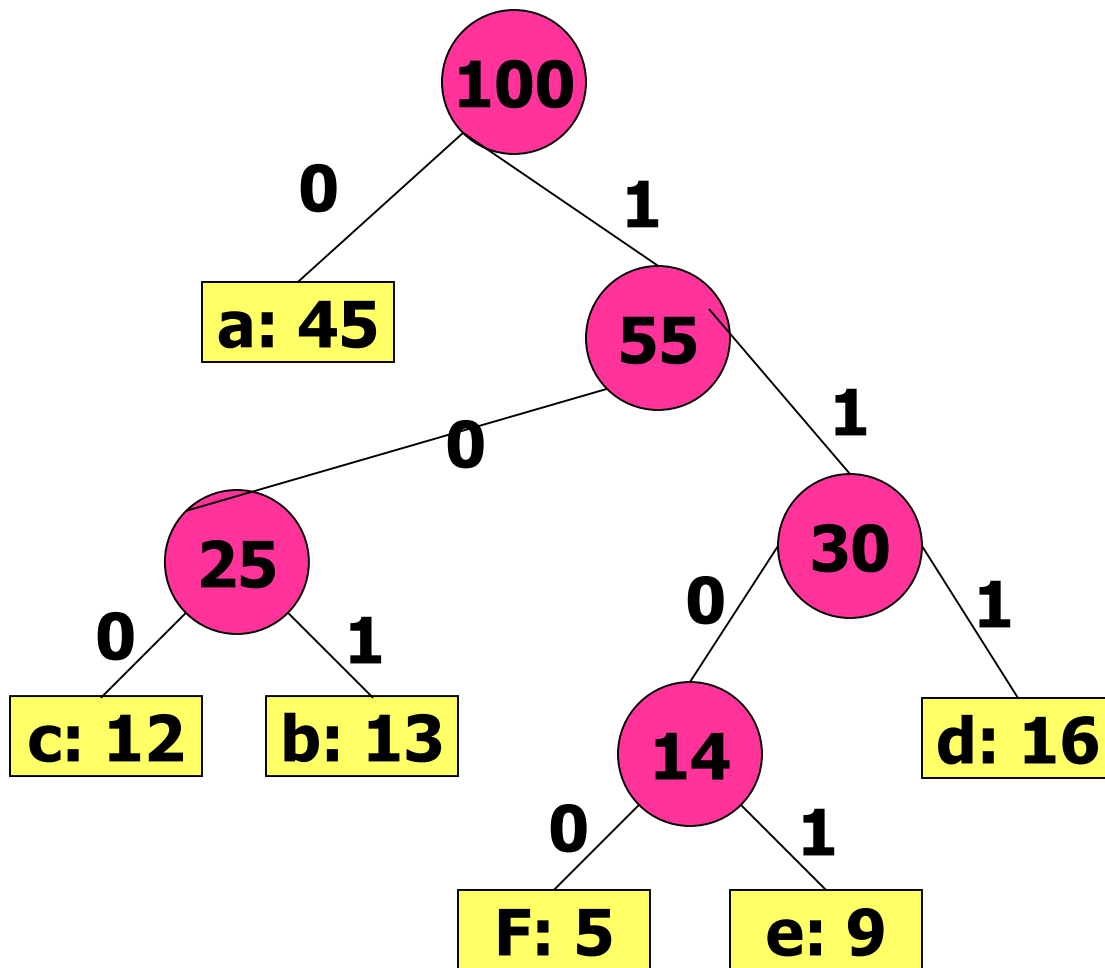
b: 13

a: 45



例子(续4)





字符	a	b	C	d	e	F
频率	45	13	12	16	9	5
变长码	0	101	100	111	1101	1100

哈夫曼编码

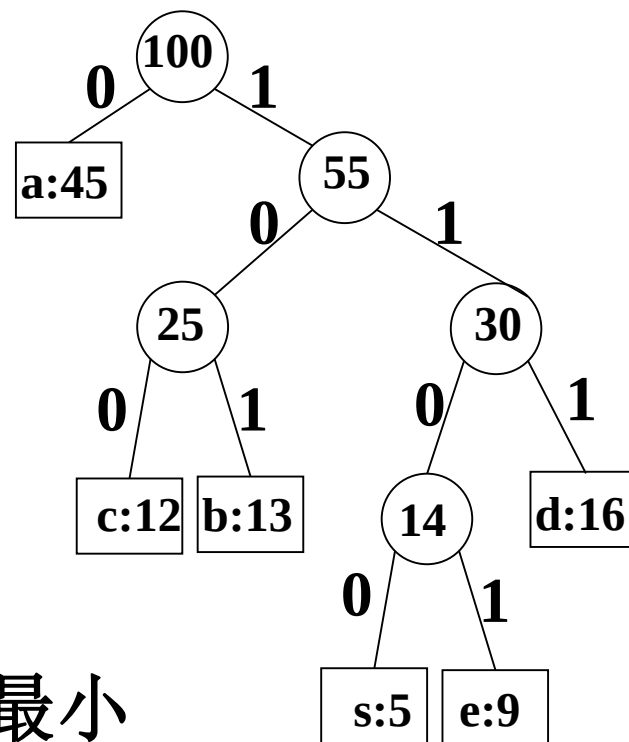
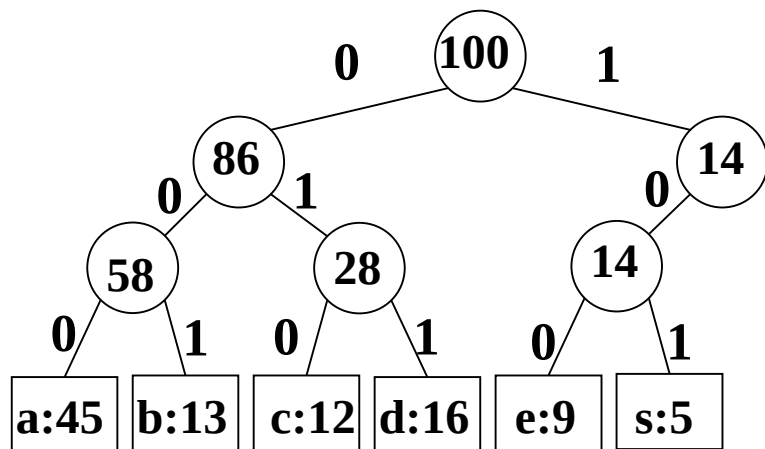
3、哈夫曼算法的正确性

要证明哈夫曼算法的正确性，只要证明最优前缀码问题具有贪心选择性质和最优子结构性。

(1) 贪心选择性质

(2) 最优子结构性

确定优化目标



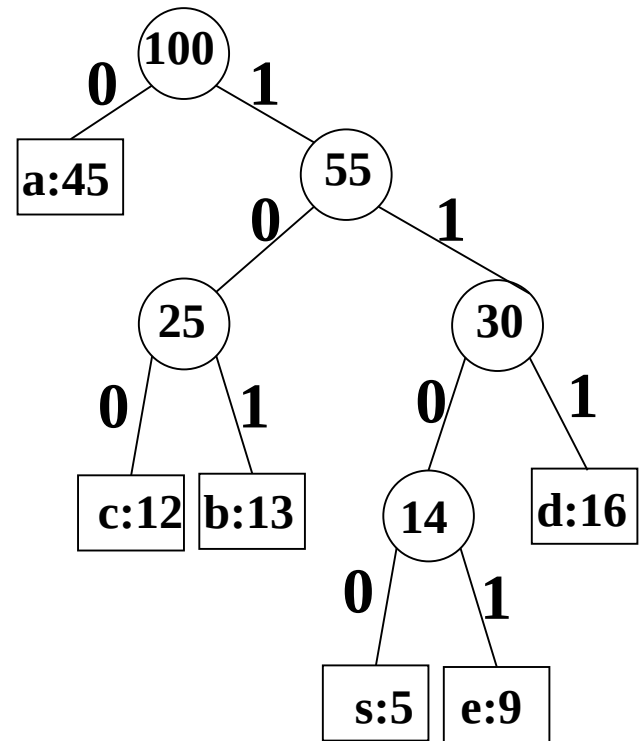
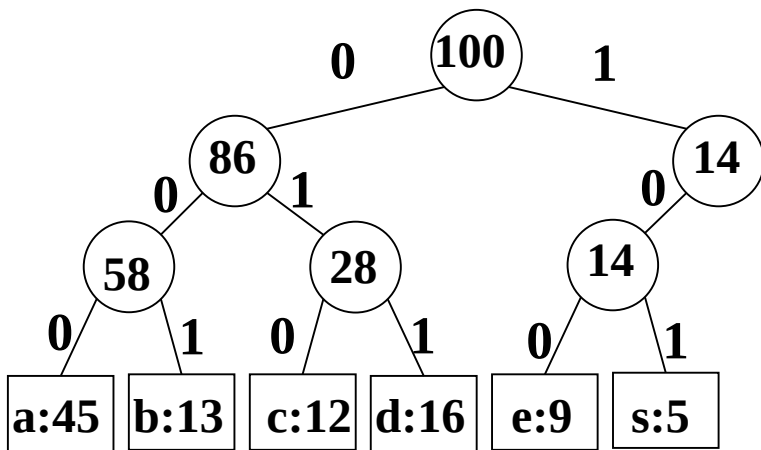
• 叶节点深度 = 编码长度

• **优化目标:** 叶节点频率乘深度的和最小

$$\min_T \sum_{c \in C} f(c) d_T(c)$$

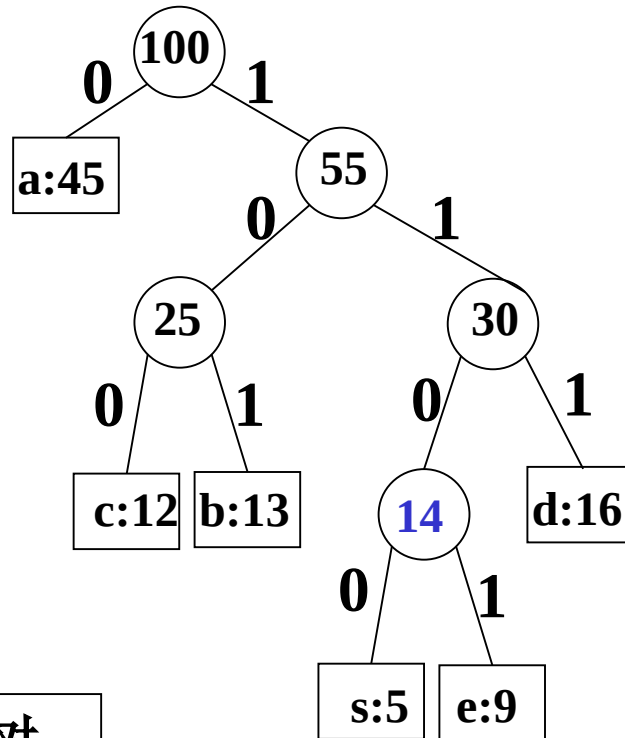
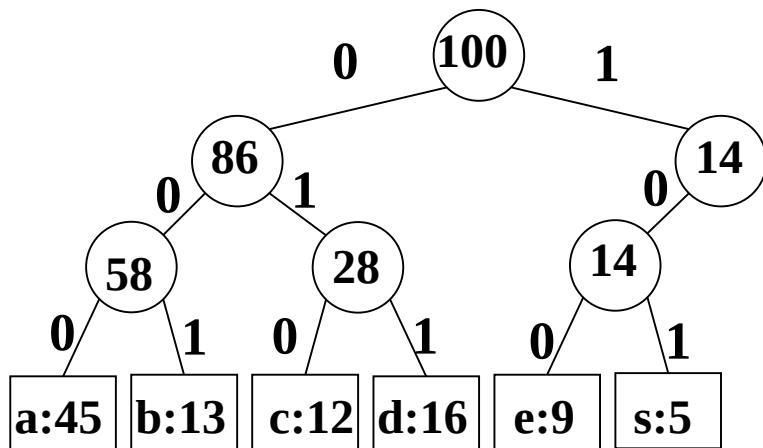
T为编码树, C是叶节点集, $f(c)$ 是c的频率, $d_T(c)$ 是c在T上的深度

好编码的特点



- 大频率编码短
- 小频率编码长
- 正则二叉树
- 相对平衡

贪心选择性质



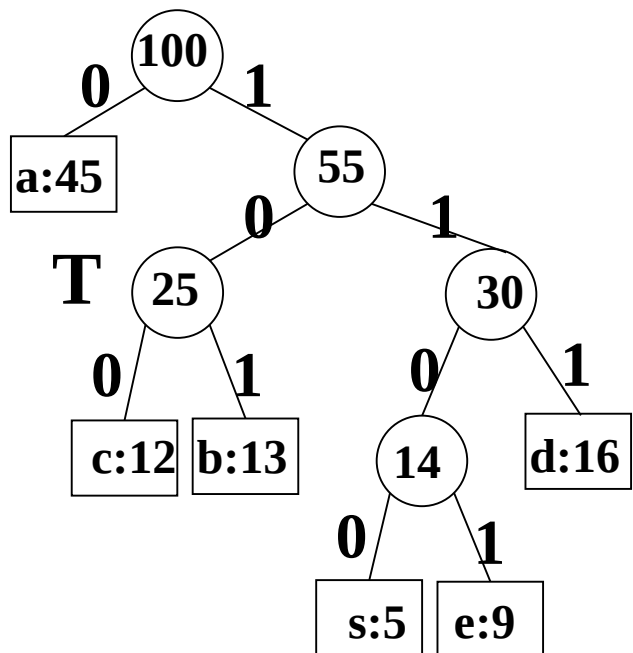
$$\min_T \sum_{c \in C} f(c) d_T(c)$$

正则二叉树
小频率编码长

⇒ 存在最优解其频率最小两叶节点深度最大

⇒ 存在最优解其频率最小两叶节点是兄弟

最优子结构性质



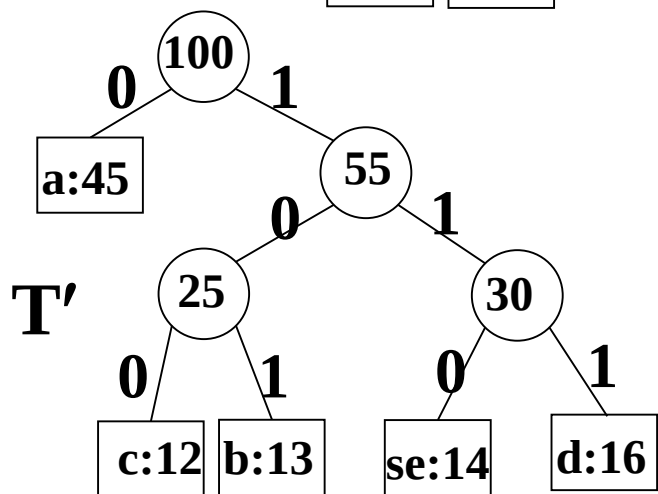
$$H(T) = \sum_{c \in C} f(c) d_T(c)$$

设T两叶节点se是兄弟

- 将 $f(s)+f(e)$ 赋值给se父节点
- 删叶节点se得子树T'

$$H(T) = H(T') + f(s) + f(e)$$

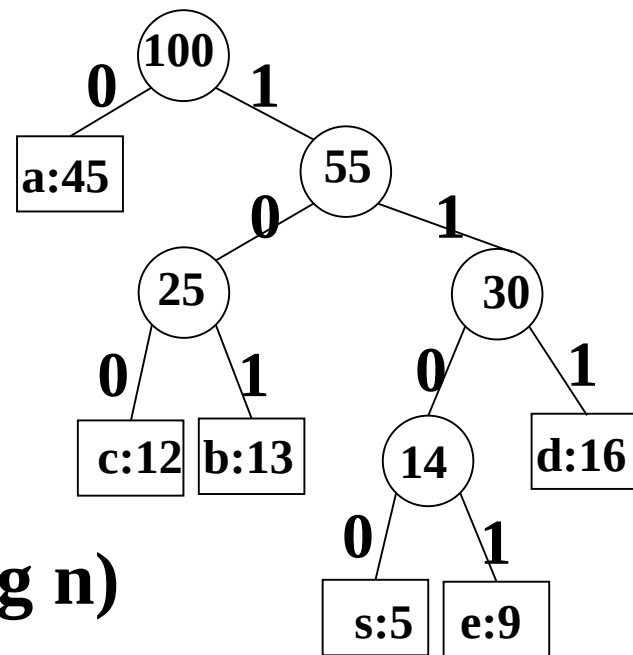
- 若T最优, 则T'最优 (OSP)



Huffman编码算法

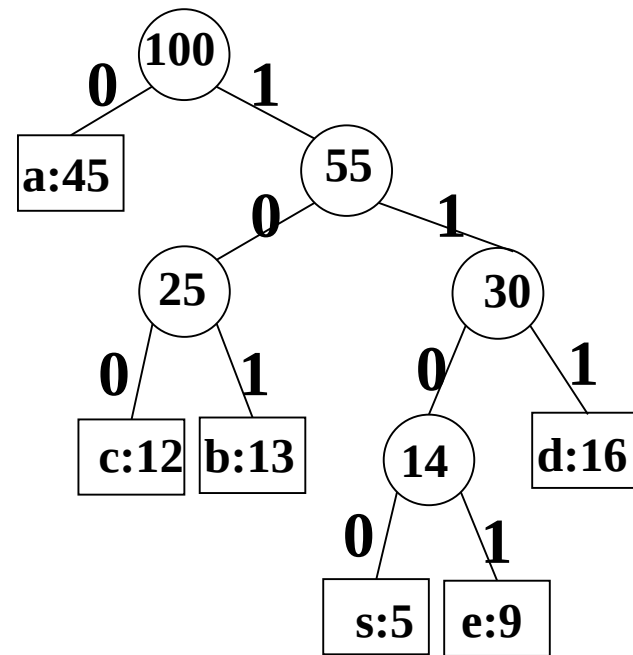
输入C[1:n], f[1:n] //字母集C和频率f

1. $Q=C$ //建优先队列Q, $O(n)$
2. 对 $i = 1:n-1$ //循环n-1次
3. 分配新节点z
4. 取Q中f最小x作为z的左孩子// $O(\log n)$
5. 取Q中f最小y作为z的右孩子// $O(\log n)$
6. $f[z] = f[x] + f[y]$
7. 将z插入Q中 // $O(\log n)$
8. 取出Q中节点作为树根



Huffman编码小结

- 输入: $C[1:n], f[1:n]$
- 输出: C 的最优前缀码
- 有贪心算法的两个基本要素
贪心选择性质 和 OSP
- 自顶向下计算?
- 正确性证明一般过程:
贪心选择+OSP+数学归纳法
子问题与原问题类似, 相对独立
子问题的最优解和贪心选择联合得整体最优解



哈夫曼编码贪心选择性质

Let a and b be two characters that are sibling leaves of maximum depth in T . Without loss of generality, we assume that $f[a] \leq f[b]$ and $f[x] \leq f[y]$. Since $f[x]$ and $f[y]$ are the two lowest leaf frequencies, in order, and $f[a]$ and $f[b]$ are two arbitrary frequencies, in order, we have $f[x] \leq f[a]$ and $f[y] \leq f[b]$. As shown in [Figure 16.6](#), we exchange the positions in T of a and x to produce a tree T' , and then we exchange the positions in T' of b and y to produce a tree T'' . By equation [\(16.5\)](#), the difference in cost between T and T' is

$$\begin{aligned} B(T) - B(T') &= \sum_{c \in C} f(c)d_T(c) - \sum_{c \in C} f(c)d_{T'}(c) \\ &= f[x]d_T(x) + f[a]d_T(a) - f[x]d_{T'}(x) - f[a]d_{T'}(a) \\ &= f[x]d_T(x) + f[a]d_T(a) - f[x]d_T(a) - f[a]d_T(x) \\ &= (f[a] - f[x])(d_T(a) - d_T(x)) \\ &\geq 0, \end{aligned}$$

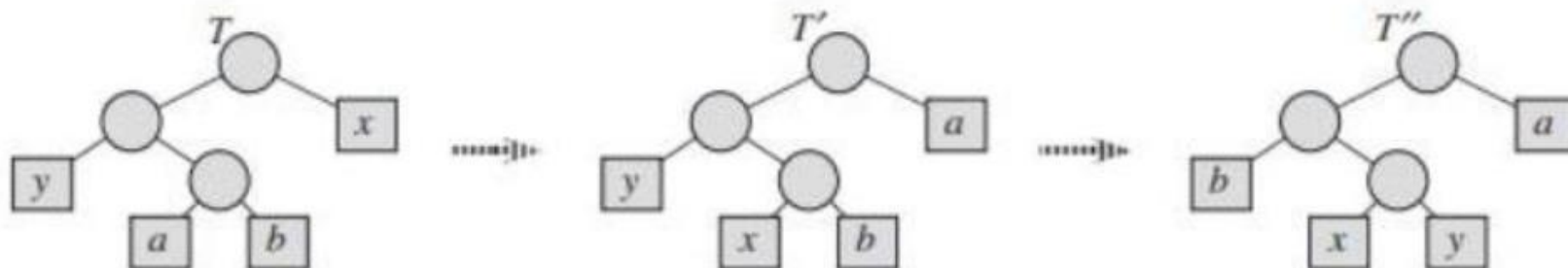
because both $f[a] - f[x]$ and $d_T(a) - d_T(x)$ are nonnegative. More specifically, $f[a] - f[x]$ is nonnegative because x is a minimum-frequency leaf, and $d_T(a) - d_T(x)$ is nonnegative because a is a leaf of maximum depth in T . Similarly, exchanging y and b does not increase

哈夫曼编码贪心选择性质

哈夫曼树的 ** 代价（带权路径长度） ** 定义为：

$$B(T) = \sum_{c \in C} f(c) \cdot d_T(c)$$

- C : 所有字符的集合;
- $f(c)$: 字符 c 的频率;
- $d_T(c)$: 字符 c 在树 T 中的深度（即编码长度）。



设 a 和 b 是树 T 中深度最大的一对兄弟叶子节点。不失一般性，我们假设 $f[a] \leq f[b]$ 且 $f[x] \leq f[y]$ 。由于 $f[x]$ 和 $f[y]$ 是按升序排列的两个频率最低的叶子节点频率，而 $f[a]$ 和 $f[b]$ 是任意顺序的两个频率，因此有 $f[x] \leq f[a]$ 且 $f[y] \leq f[b]$ 。

如图 16.6 所示，我们先在树 T 中交换 a 和 x 的位置，得到树 T' ；再在树 T' 中交换 b 和 y 的位置，得到树 T'' 。根据公式 (16.5)，树 T 与 T' 的代价差为：

$$\begin{aligned} B(T) - B(T') &= \sum_{c \in C} f(c)d_T(c) - \sum_{c \in C} f(c)d_{T'}(c) \\ &= f[x]d_T(x) + f[a]d_T(a) - f[x]d_{T'}(x) - f[a]d_{T'}(a) \\ &= f[x]d_T(x) + f[a]d_T(a) - f[x]d_T(a) - f[a]d_T(x) \\ &= (f[a] - f[x])(d_T(a) - d_T(x)) \\ &\geq 0 \end{aligned}$$

这是因为 $f[a] - f[x]$ 和 $d_T(a) - d_T(x)$ 均为非负数。具体来说：

- $f[a] - f[x]$ 非负，是因为 x 是频率最低的叶子节点；
- $d_T(a) - d_T(x)$ 非负，是因为 a 是树 T 中深度最大的叶子节点。

同理，交换 y 和 b 的位置也不会使代价增加。

第四章 贪心算法

Greedy

!!贪心不一定正确(0-1背包), 需要证明

4.1 活动安排问题

4.2 贪心算法的基本要素 (分数背包)

4.3 最优装载

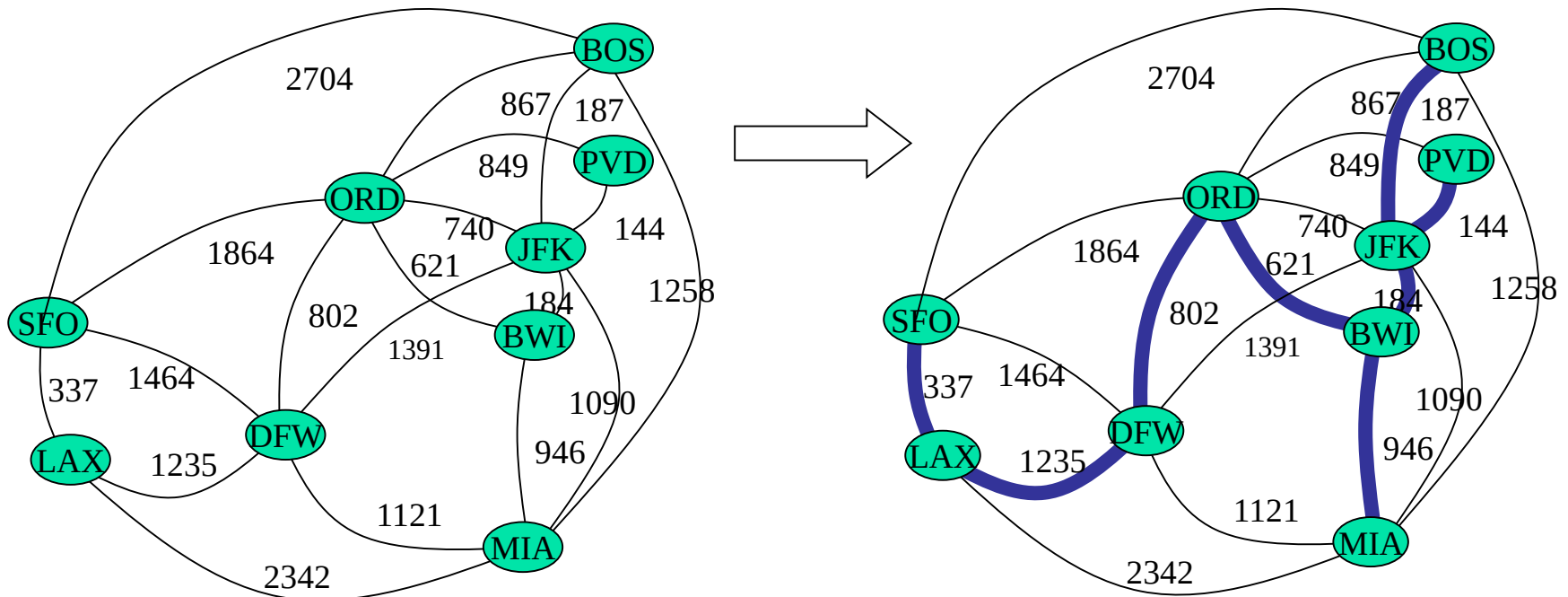
4.4 哈夫曼编码

4.6 最小生成树

应用：最小生成树

最小生成树(MST[王]P103)

- 无向连通带权图 $G=(V,E,w)$.
- G 的生成树是 G 的包含所有顶点的一颗子树
- 若 G 的生成树 T' 在所有 G 的生成树中各边权总和最小, 则 T' 称为 G 的最小生成树(MST, minimum spanning tree)



问题

- 如何构造最小生成树？
- 有哪些构造最小生成树的方法？贪心选择依据是什么？

最小生成树

用贪心算法设计策略可以设计出构造最小生成树的有效算法。本节介绍的构造最小生成树的Prim算法和Kruskal克鲁斯卡尔算法都可以看作是应用贪心算法设计策略的例子。

尽管这2个算法做贪心选择的方式不同，它们都利用了下面的最小生成树性质：

设 $G=(V,E)$ 是连通带权图， U 是 V 的真子集。如果 $(u,v) \in E$ ，且 $u \in U$ ， $v \in V-U$ ，且在所有这样的边中， (u,v) 的权 $c[u][v]$ 最小，那么一定存在 G 的一棵最小生成树，它以 (u,v) 为其中一条边。这个性质有时也称为MST性质。

最小生成树

Prim最小生成树算法

- Prim在1957年提出一种算法，这种算法特别适用于边数相对较多，即比较接近于完全图的图。

- 基本思想

此算法是按逐个将顶点连通的步骤进行的，它只需采用一个顶点集合。这个集合开始时是空集，以后将已连通的顶点陆续加入到集合中去，到全部顶点都加入到集合中了，就得到所需的生成树。

最小生成树

Prim算法

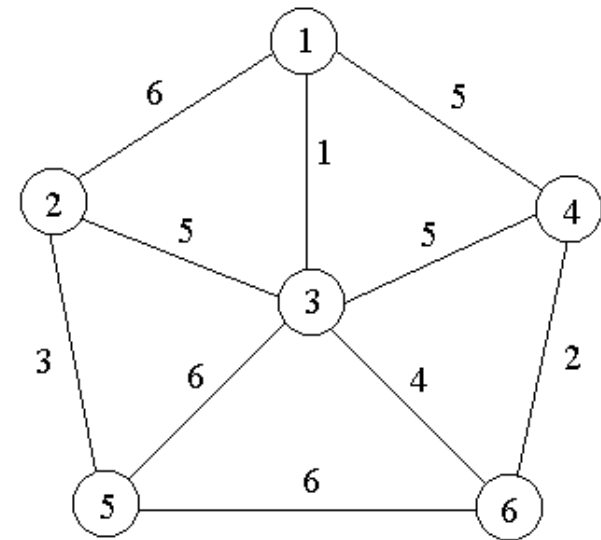
- 设 $G=(V,E)$ 是连通带权图， $V=\{1,2,\dots,n\}$ 。
- 构造 G 的最小生成树的Prim算法的**基本思想**是：首先置 $S=\{1\}$ ，然后，只要 S 是 V 的真子集，就作如下的**贪心选择**：选取满足条件 $i \in S$ ， $j \in V-S$ ，且 $c[i][j]$ 最小的边，将顶点 j 添加到 S 中。这个过程一直进行到 $S=V$ 时为止。
- 在这个过程中选取到的所有边恰好构成 G 的一棵**最小生成树**。

最小生成树

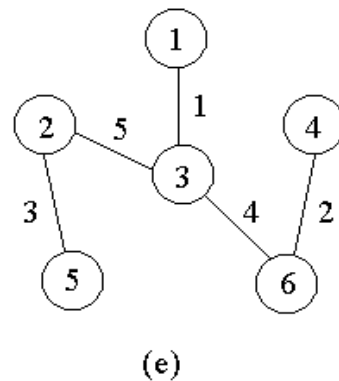
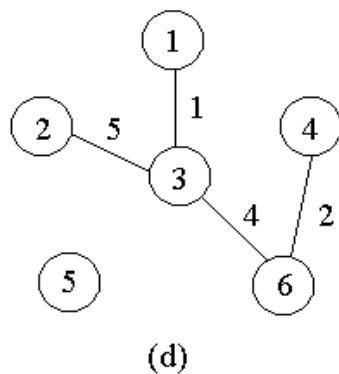
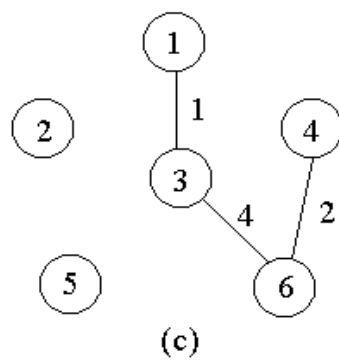
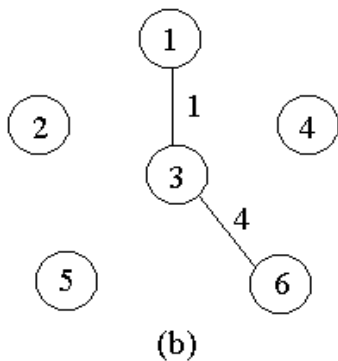
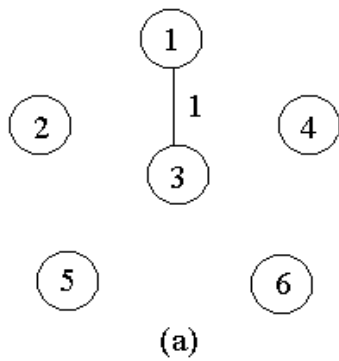
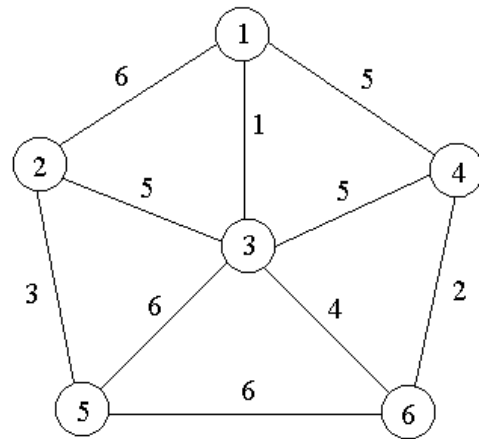
- 问题：Prim算法选取的边是否会产生回路？
- 由于Prim算法中每次选取的边两端总是一个已连通顶点和一个未连通顶点，故这个边选取后一定能将该未连通点连通而又保证不会形成回路。

最小生成树

- 结论：利用最小生成树性质和数学归纳法容易证明，上述算法中的边集合T始终包含G的某棵最小生成树中的边。因此，在算法结束时，T中的所有边构成G的一棵最小生成树。
- 例如，对于右图中的带权图，按Prim算法选取边的过程如下。



最小生成树



最小生成树

- 在上述Prim算法中，还应当考虑如何有效地找出满足条件 $i \in S, j \in V-S$ ，且权 $c[i][j]$ 最小的边 (i, j) 。实现这个目的的较简单的办法是设置2个数组closest(记录j在s中的邻接结点)和lowcost($=c[j][closest[j]]$)。
- 在Prim算法执行过程中，先找出V-S中使lowcost值最小的顶点j，然后根据数组closest选取边 $(j, closest[j])$ ，最后将j添加到S中，并对closest和lowcost作必要的修改。

用这个办法实现的Prim算法所需的计算时间为 $O(n^2)$

最小生成树Kruskal算法

Kruskal算法构造G的最小生成树的基本思想是，首先将G的n个顶点看成n个孤立的连通分支。将所有的边按权从小到大排序。然后从第一条边开始，依边权递增的顺序查看每一条边，并按下述方法连接2个不同的连通分支：当查看到第k条边 (v,w) 时，

- (1) 如果端点v和w分别是当前2个不同的连通分支T1和T2中的顶点时，就用边 (v,w) 将T1和T2连接成一个连通分支，然后继续查看第k+1条边；
- (2) 如果端点v和w在当前的同一个连通分支中，就直接再查看第k+2条边。这个过程一直进行到只剩下一个连通分支时为止。

最小生成树

Kruskal最小生成树算法

//在一个具有 n 个顶点的网络中找棵最小生成树

令 E 为网络中边的集合

令 T 为所选边的集合, 初始化 T 为空集

while(E 不是空集 && T 中元素个数不等于 $n-1$){

 令 (u,v) 为 E 中最小代价的边;

$E=E-(u,v)$; //从中删除该边;

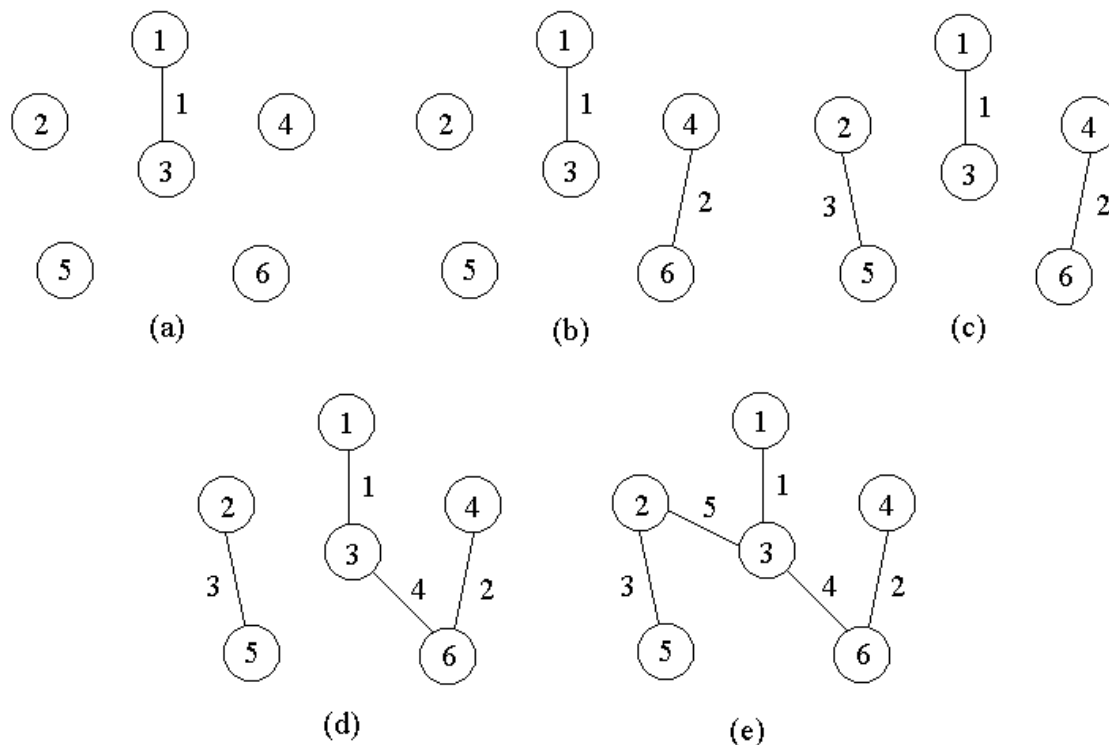
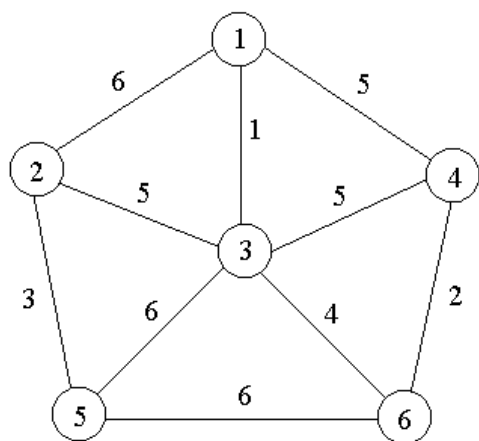
if((u,v)加入 T 中不会产生环)

 将 (u,v) 加入 T ;

}

最小生成树

例如，对前面的连通带权图，按Kruskal算法顺序得到的最小生成树上的边如下图所示。



最小生成树

当图的边数为 e 时，Kruskal算法所需的计算时间是 $O(e \log e)$ 。

当 $e = \Omega(n^2)$ 时，Kruskal算法比Prim算法差，
当 $e = o(n^2)$ 时，Kruskal算法比Prim算法好得多。

并查集算法(Make-set, Find-set)

$p[x]$: x 的父亲; $\text{rank}[x]$: x 的阶; $\text{Find}(x)$: 找 x 的根

Make(x)

1 $p[x] = x$

2 $\text{rank}[x] = 0$

Union(x, y)

1 $\text{Link}(\text{Find}(x), \text{Find}(y))$

Find(x)

1 若 $x \neq p[x]$, 则

2 $p[x] = \text{Find}(p[x])$

3 返回 $p[x]$

路径压缩(pc)技术

Link(x, y) // 合并两树根

1 若 $\text{rank}[x] > \text{rank}[y]$

2 则 $p[y] = x$

3 否则 $p[x] = y$

4 若 $\text{rank}[x] = \text{rank}[y]$

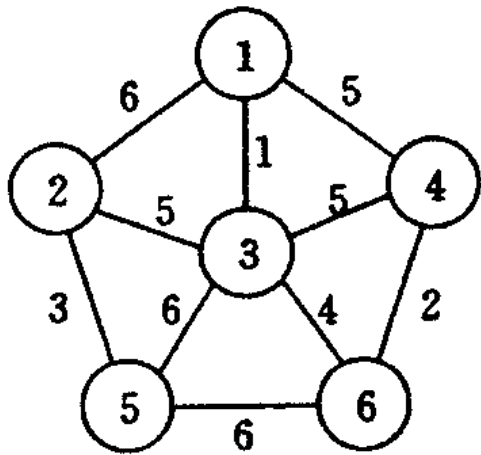
5 则 $\text{rank}[y] = \text{rank}[y] + 1$

加入并查集结构的Kruskal算法

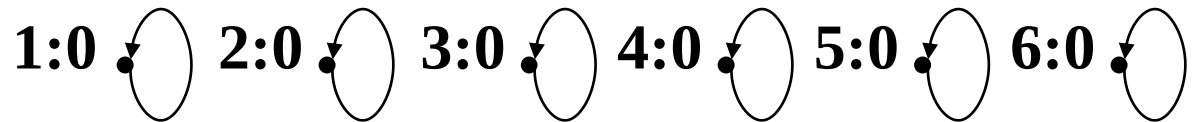
1. A为空, Q=E按边权升序排列, 每个点是一颗树
 2. 当Q非空
 3. 顺序取Q中边(u,v)
 4. 若u,v在不同树中, 则添(u,v)到A, 合并u,v所在树,
 5. 输出A
-

1. A为空, Q=E按边权升序排列, $\forall x$ Make(x)
2. 当Q非空
3. 顺序取Q中边(u,v)
4. 若Find(u)≠Find(v), 则添(u,v)到A, Union(u,v),
5. 输出A

Kruskal: 取边, 查找, 合并

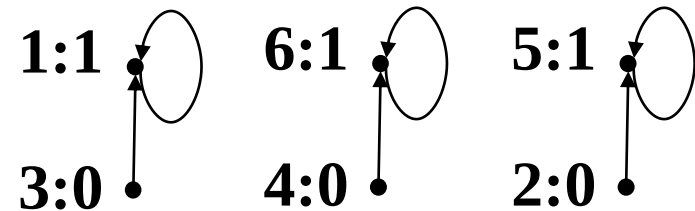


$Q = \{31, 46, 25, 36, 34, 23, 14, 12, 35, 56\}$

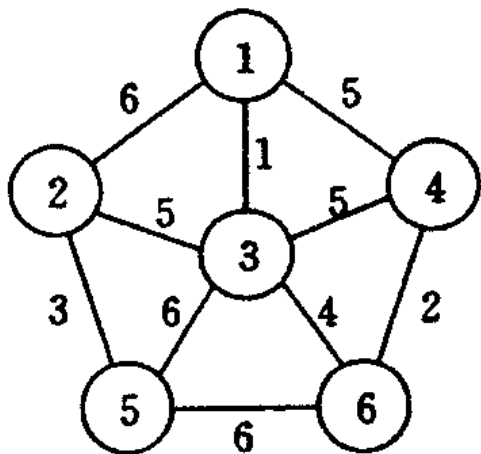


查找合并

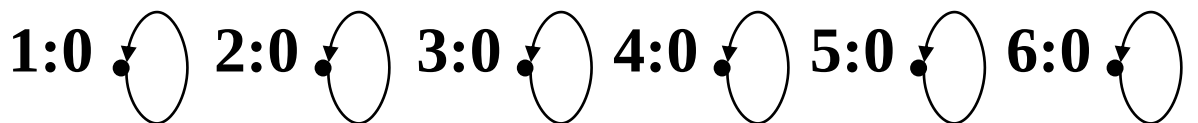
$(3,1) (4,6) (2,5)$



Kruskal: 取边, 查找, 合并

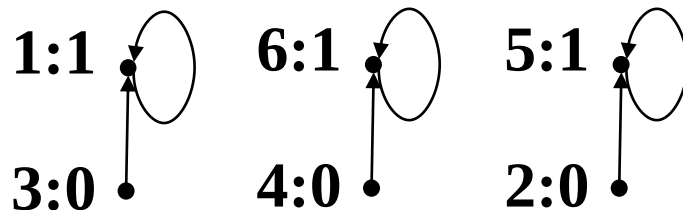


$Q = \{31, 46, 25, 36, 34, 23, 14, 12, 35, 56\}$

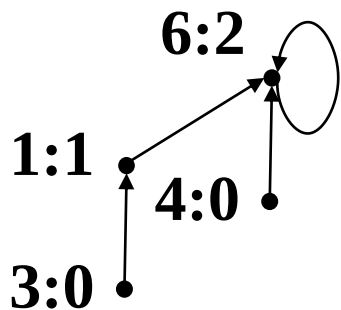


查找合并

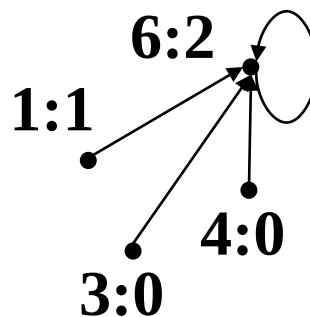
(3,1) (4,6) (2,5)



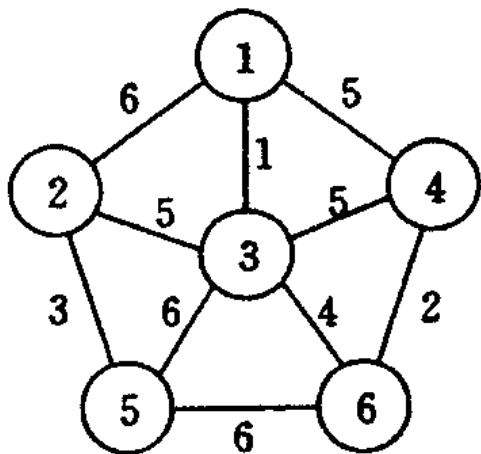
查并(3,6)



查(3,4)



Kruskal: 取边, 查找, 合并



$Q=\{31, 46, 25, 36, 34, 23, 14, 12, 35, 56\}$

1:0 2:0 3:0 4:0 5:0 6:0

查找合并

(3,1) (4,6) (2,5)

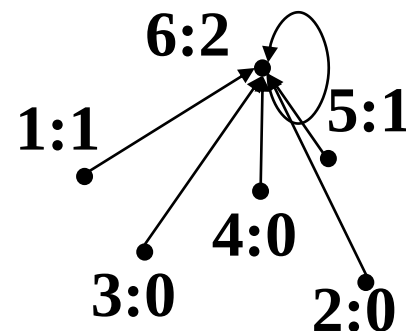
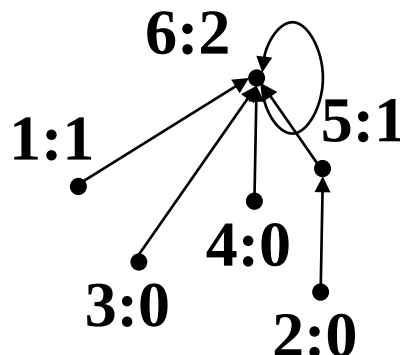
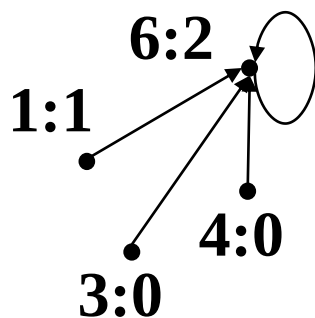
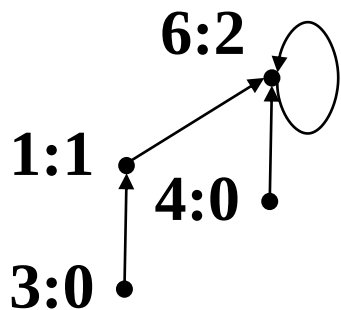
1:1 6:1 5:1
3:0 4:0 2:0

查并(3,6)

查(3,4)

查并(2,3)

查(1,2)



排序之车间作业计划

一：一台机器、n个零件,使得各加工零件在车间里停留的平均时间最短。

零件	加工时间
1	1.8
2	2.0
3	0.5
4	0.9
5	1.3
6	1.5

若按此顺序:

$$(1.8+3.8+4.3+5.2+6.5+8)/6=4.9$$

实际上最短: 3, 4, 5, 6, 1, 2

$$(0.5+1.4+2.7+4.2+6+8)/6=3.8$$

排序之车间作业计划

二：两台机器、 n 个零件的排序

目的：使得完成全部工作的总时间最短。

排序之车间作业计划

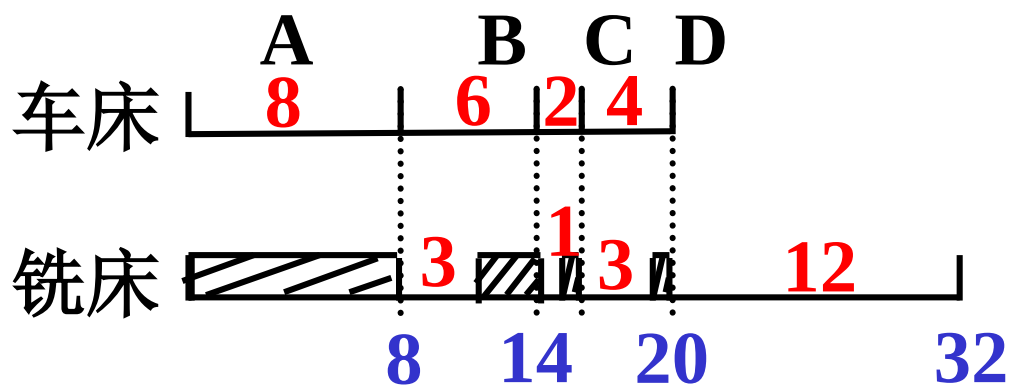
用一台车床和一台铣床加工A、B、C、D四个零件。每个零件都需要先用车床加工，再用铣床加工。车床与铣床加工每个零件所需的工时：

工时（小时）	A	B	C	D
车床	8	6	2	4
铣床	3	1	3	12

若以A、B、C、D零件顺序安排加工，则共需32小时。适当调整零件加工顺序，可产生不同实施方案，我们称可使所需总工时最短的方案为最优方案。

工时（小时）	A	B	C	D
车床	8	6	2	4
铣床	3	1	3	12

以A、B、C、D零件顺序安排加工，则共需32小时。



基本方法：

在第一台机器上加工时间越少的零件越早加工；

在第二台机器上加工时间越少的零件越晚加工；

排序之车间作业计划

工时（小时）	A	B	C	D
车床	8	6	2	4
铣床	3	1	3	12

(1)

第一个:

第二个:

第三个:

第四个: **B**

(2)

第一个: **C**

第二个:

第三个:

第四个: **B**

(3)

第一个: **C**

第二个:

第三个: **A**

第四个: **B**

(4)

第一个: **C**

第二个: **D**

第三个: **A**

第四个: **B**

CDAB: 22

教室安排问题

- 假设要用很多个教室对一组活动进行调度。我们希望使用尽可能少的教室来调度所有的活动。
- 例如，有如下活动的开始和结束时间。
- 1 3
- 2 4
- 4 7
- 3 5
- 1 4
- 统一排序

教室安排问题

- 1 3
- 2 4
- 4 7
- 3 5
- 1 4
- 分别排序
- 若开始<结束, 则count++, 指针j后移

1 1 2 3 4

- 3 4 4 5 7

- 1 3
- 2 4
- 4 7
- 3 5
- 1 4
- 只按开始时间排序(1, 3) (1, 4) (2, 4) (3, 5) (4, 7)

[illegible]

本章小结

!!贪心不一定正确(0-1背包), 需要证明

4.1 活动安排问题

4.2 贪心算法的基本要素（分数背包）

4.3 最优装载

4.4 哈夫曼编码

4.6 最小生成树